Deception Based Security Mechanism

Cybersecurity and Network Security Internship Assessment

Submitted By : Morabad Riya Shivanand

Date : 02 April 2026

Table of Contents

1. Introduction 3
2. System Design 3
3. Implementation 4
4. Testing and Results 9
5. Analysis of Results 10
6. Conclusion 11
7. Refernces 11

1. Introduction

Modern cyber-attacks frequently include efforts by attackers to access systems or sensitive information without proper authorization. Conventional security systems emphasize intrusion prevention through authentication, firewalls, and monitoring tools. However, security methods based on deception add another layer of defense by intentionally setting up fake resources to lure attackers and enable security teams to monitor unusual behavior.

A honeypot system was used in this project to implement a deception-based security mechanism. A honeypot is a fake system created to imitate real services while discreetly observing and recording attacker activities. In this project, a counterfeit online banking login page was created to serve as a lure. Any interaction with the page is considered suspicious since the page does not offer legitimate functionality.

The system logs data including the visitor's IP address, browser information, the username and password inputted, and the times of interactions. These logs aid in recognizing possible malicious activities and show how deception technologies can support intrusion detection and threat surveillance.

2. System Design

The security system based on deception was created with a honeypot structure that includes three primary elements:

1. Fake Login Interface
A realistic bank login page was designed to mimic an online banking portal. The page features username and password input fields, giving it a genuine appearance to users. However, the system does not verify the user or provide access.

2. Monitoring and Logging System
Whenever a visitor engages with the login page, the system records key information such as:
- Visitor's IP Address

- Browser details (User Agent)

- Entered username and password

- Timestamp of the interaction

- Type of event (page visit or login attempt)
This information is saved in a structured JSON log file.

3. Alert and Monitoring Dashboard

An administrative dashboard enables administrators to view captured suspicious activities. The dashboard shows all recorded events and emphasizes questionable login attempts.

This design guarantees that every interaction with the fake login interface is logged and review.

3.Implementation

The system was developed utilizing the Python programming language along with the Flask web framework. Flask was selected due to its lightweight framework, which is ideal for developing web applications and managing HTTP requests.

Various Python libraries were utilized during the implementation:

- Flask employed to establish the web server and manage user requests.
- JSON is utilized for storing log entries in an organized format.
- Date time utilized to create timestamps for events.
- OS utilized for managing the log file.

The honeypot system can quickly capture and process suspicious interactions thanks to the way these libraries work together. Flask takes care of HTTP requests from users who go to the fake login page, and the backend Python code processes the data that is sent and logs the event. The JSON module is used to save captured data in a way that makes it easy to find and analyse later. The datetime module makes sure that each recorded event has the right timestamp, which helps keep track of when suspicious activities happen. The OS module is also used to see if the log file already exists and to handle file operations when new log entries are added.

These parts work together to make a simple but effective system for keeping an eye on how people interact with the fake login screen. The implementation shows how to make a lightweight honeypot system that can find and record potentially malicious behaviour by combining handling web requests, storing structured data, and adding timestamps to events.

Fake Login Page

A counterfeit bank login page was designed utilizing HTML and CSS. The interface includes spaces for inputting a username and password, as well as a login button. The design mimics a standard online banking login page to give the honeypot a legitimate appearance.

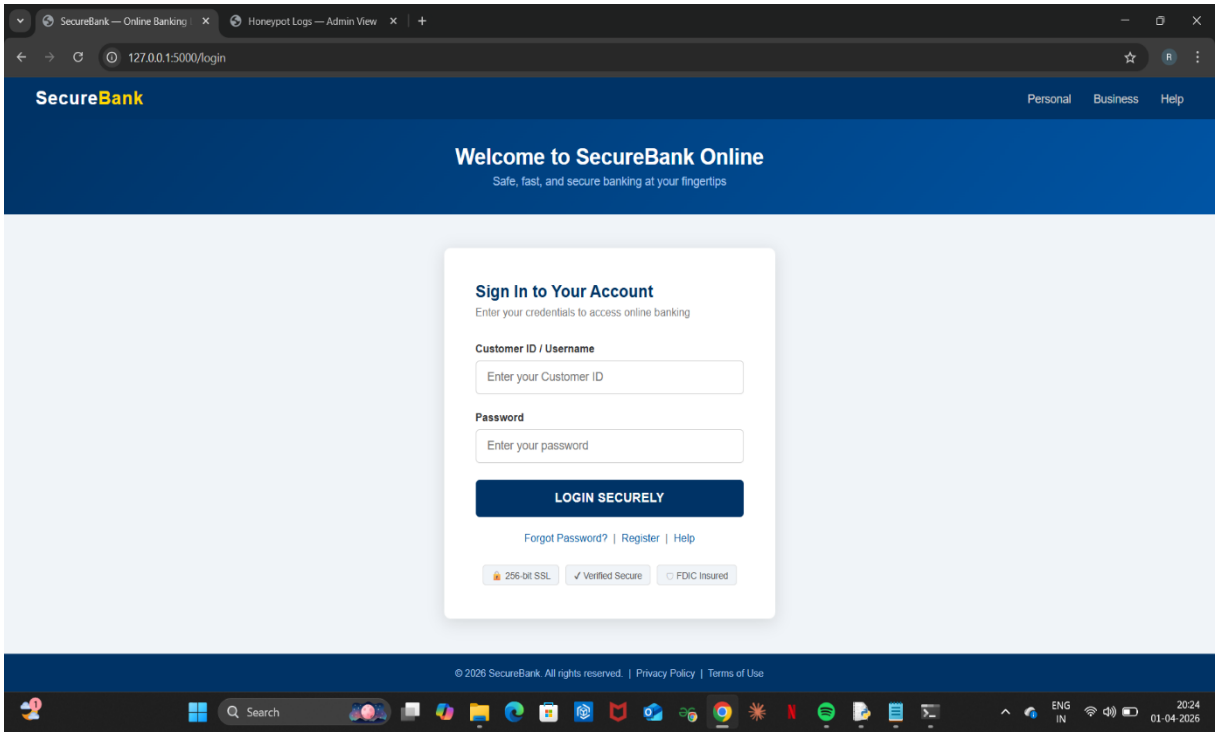


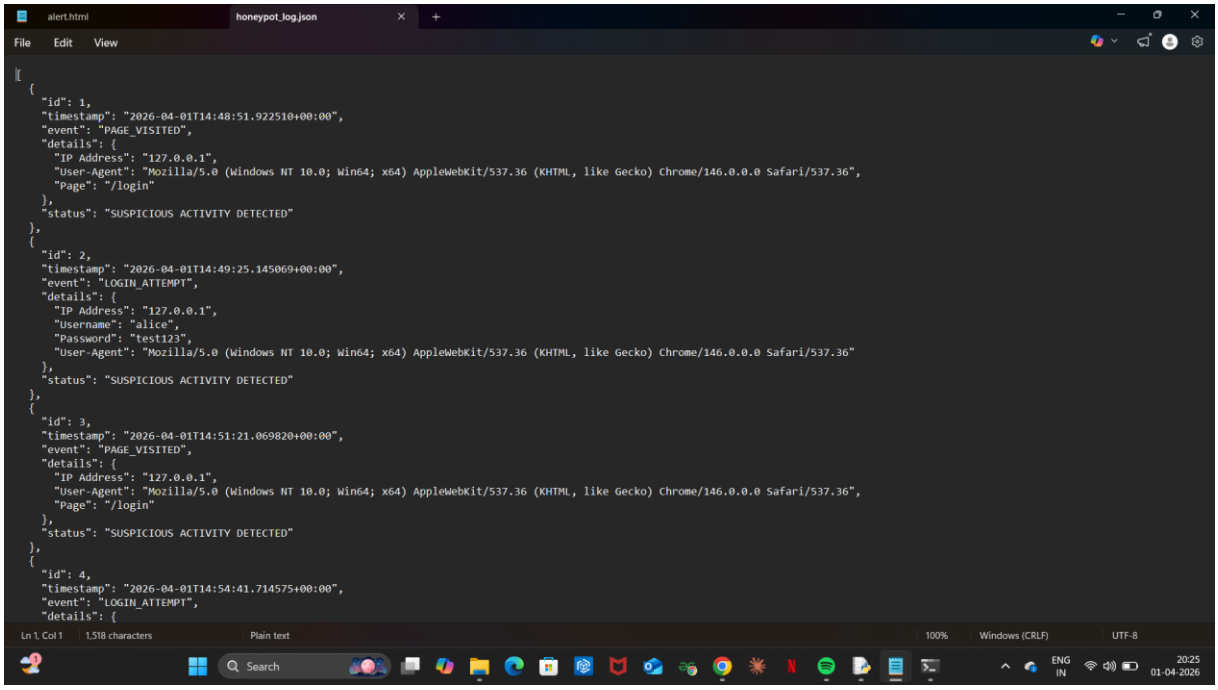
Figure 1: Fake SecureBank Login Page (Honeypot Trap)

Logging Events

The system logs any time a user goes to the login page or enters their login information as a suspicious event. There is a log entry that says:

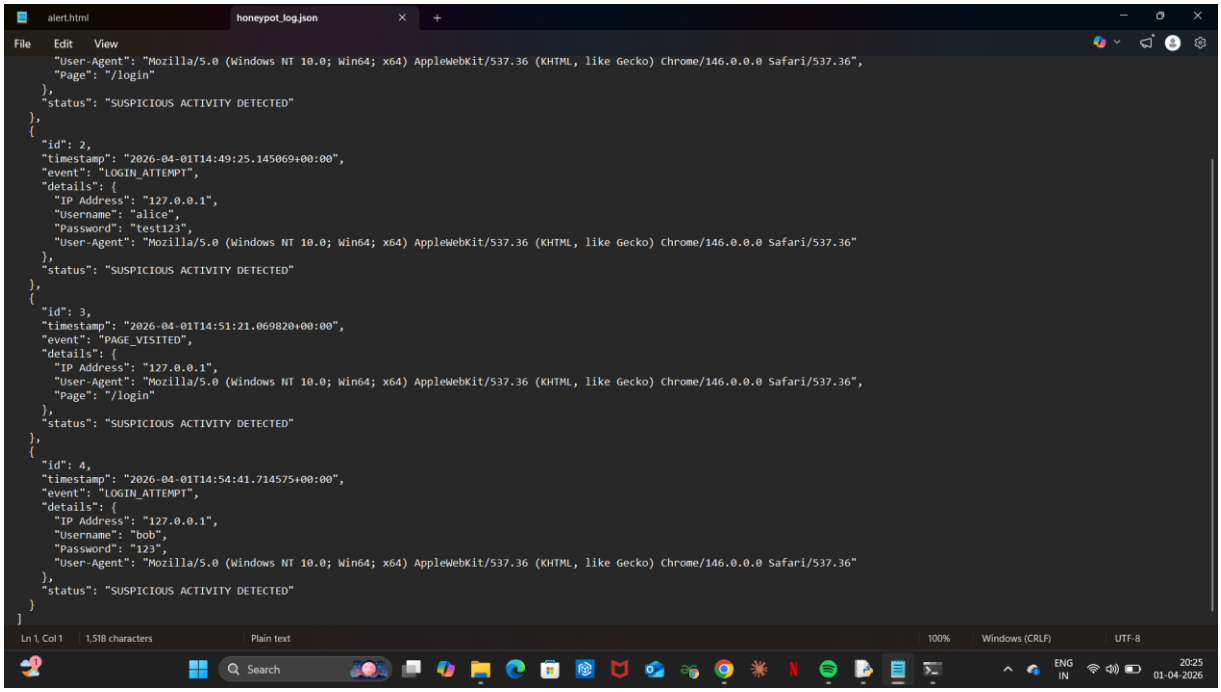
Type of event (PAGE_VISITED or LOGIN_ATTEMPT)
Time stamp and IP address of the user
You typed in your username and password.
User agent for the browser

The data is saved in a JSON file called honeypot_log.json.




```
[
  {
    "id": 1,
    "timestamp": "2026-04-01T14:48:51.922510+00:00",
    "event": "PAGE_VISITED",
    "details": {
      "IP Address": "127.0.0.1",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36",
      "Page": "/login"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 2,
    "timestamp": "2026-04-01T14:49:25.145069+00:00",
    "event": "LOGIN_ATTEMPT",
    "details": {
      "IP Address": "127.0.0.1",
      "Username": "alice",
      "Password": "test123",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 3,
    "timestamp": "2026-04-01T14:51:21.069820+00:00",
    "event": "PAGE_VISITED",
    "details": {
      "IP Address": "127.0.0.1",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36",
      "Page": "/login"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 4,
    "timestamp": "2026-04-01T14:54:41.714575+00:00",
    "event": "LOGIN_ATTEMPT",
    "details": {
```


Figure 2: Figure 2: Honeypot Log File Showing PAGE_VISITED Event




```
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36",
      "Page": "/login"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 2,
    "timestamp": "2026-04-01T14:49:25.145069+00:00",
    "event": "LOGIN_ATTEMPT",
    "details": {
      "IP Address": "127.0.0.1",
      "Username": "alice",
      "Password": "test123",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 3,
    "timestamp": "2026-04-01T14:51:21.069820+00:00",
    "event": "PAGE_VISITED",
    "details": {
      "IP Address": "127.0.0.1",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36",
      "Page": "/login"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  },
  {
    "id": 4,
    "timestamp": "2026-04-01T14:54:41.714575+00:00",
    "event": "LOGIN_ATTEMPT",
    "details": {
      "IP Address": "127.0.0.1",
      "Username": "bob",
      "Password": "123",
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"
    },
    "status": "SUSPICIOUS ACTIVITY DETECTED"
  }
]
```


Figure 3: Honeypot Log File Showing LOGIN_ATTEMPT Event

Alert System

The system sends the user to a warning page that says "suspicious activity has been detected" after they try to log in. This shows how a security system might react to possible threats. As soon as the login form is sent, the alert page comes up. The app doesn't let the user access any real system features; instead, it sends them to a separate warning screen. This page tells the user that the login attempt was marked as suspicious and that the security monitoring system has kept track of the activity.

The backend application also saves information about the login attempt in the honeypot log file at the same time. The system keeps track of things like the user's IP address, the credentials they entered, the browser they used, and the time of the attempt. This makes sure that all suspicious interactions are recorded so they can be looked into later.

The goal of this alert system is to mimic how a real security system might react when it finds someone who isn't allowed to be there.

This alert system is meant to act like a real security system would when it sees someone trying to get in without permission. In real-world security settings, these kinds of alerts might lead to other actions, like letting administrators know, blocking the attacker's IP address, or starting more investigation.

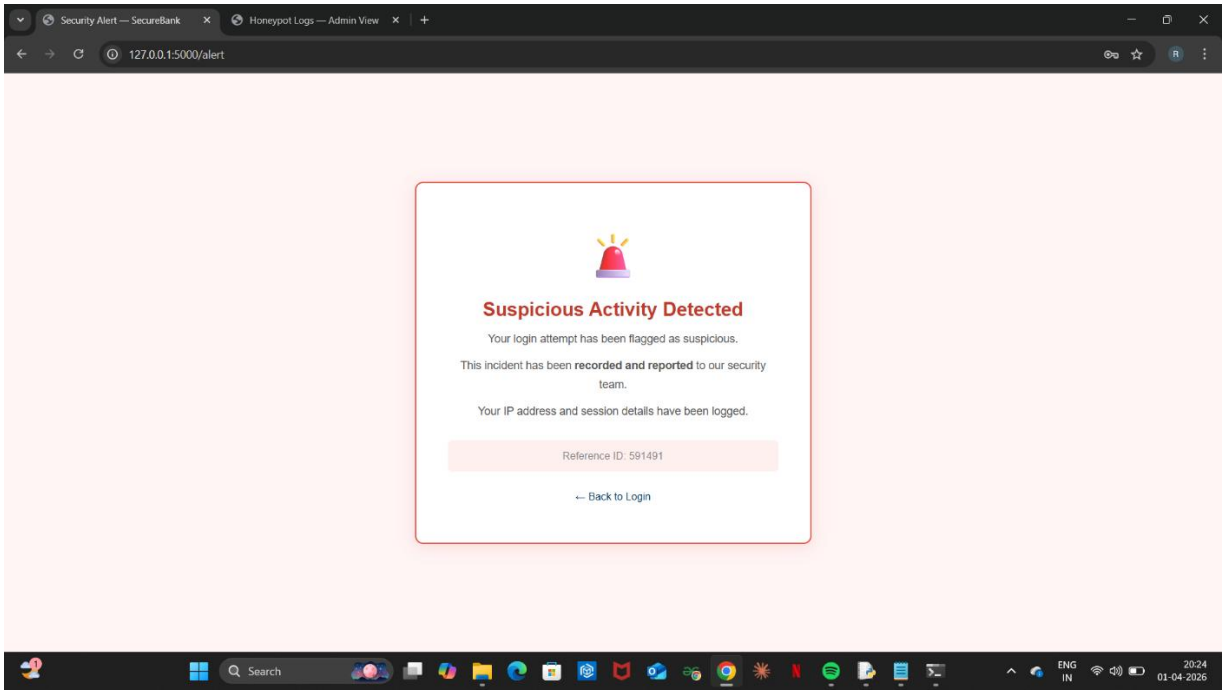


Figure 4: Suspicious Activity Alert Page Triggered After Login Attempt

Viewer for the Administrative Log

A new web page was made so that administrators could look over the interactions that the honeypot system recorded. This page is a monitoring dashboard that shows all the suspicious activities that the system has recorded in a clear and organised way.

The web interface shows information from the honeypot log file that the administrative log viewer gets. Along with the type of event, the time and date, the visitor's IP address, the username and password they entered, and the browser user-agent information, each recorded event is shown. By putting the data in this order, administrators can easily see patterns in behaviour that are suspicious.

The log viewer makes it easy for security analysts to see when someone tries to log in and what credentials they used. This information can give you a good idea of how attackers act and what they might do to try to break into your system. The system makes it easy to tell the difference between normal and suspicious activities by highlighting them.

In the real world, security operations centers use similar monitoring dashboards to keep an eye on possible threats and look at how attacks happen. This implementation is a basic prototype, but it shows how a deception-based security mechanism can be used to keep track of what attackers are doing.

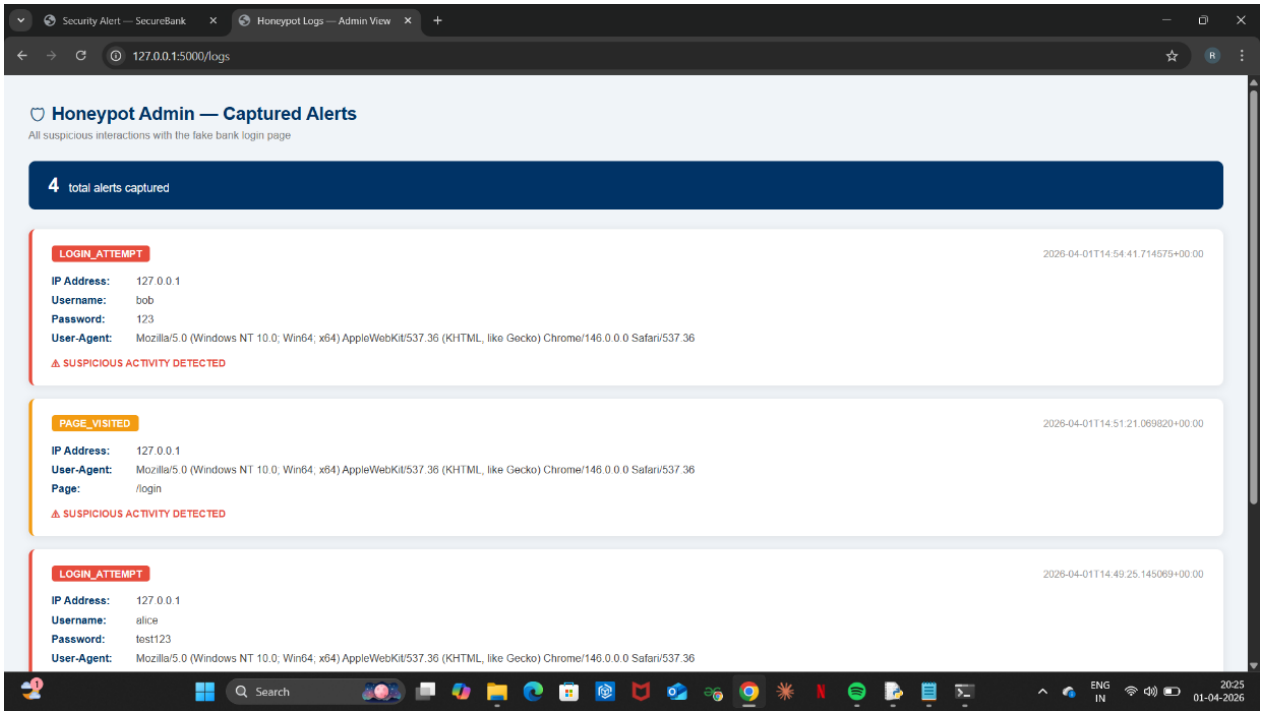


Figure 5 : Administrative Dashboard Displaying Captured Honeypot Alerts

4.Testing and Results

We tested the deception-based honeypot system by pretending to log in to the fake login page many times. The goal of the testing phase was to see if the system could find and record suspicious activity.

We first used the Flask app to start the honeypot server for testing. After the server was up and running, a web browser was used to get to the fake banking login page. The system automatically logged the visit as a PAGE_VISITED event when the login page was opened. It also saved information about the visitor, such as their IP address and browser user-agent.

Next, different usernames and passwords were typed into the login form to see what it would be like if someone tried to get in without permission. After entering their credentials, the system sent the user to a warning page that said suspicious activity had been found.

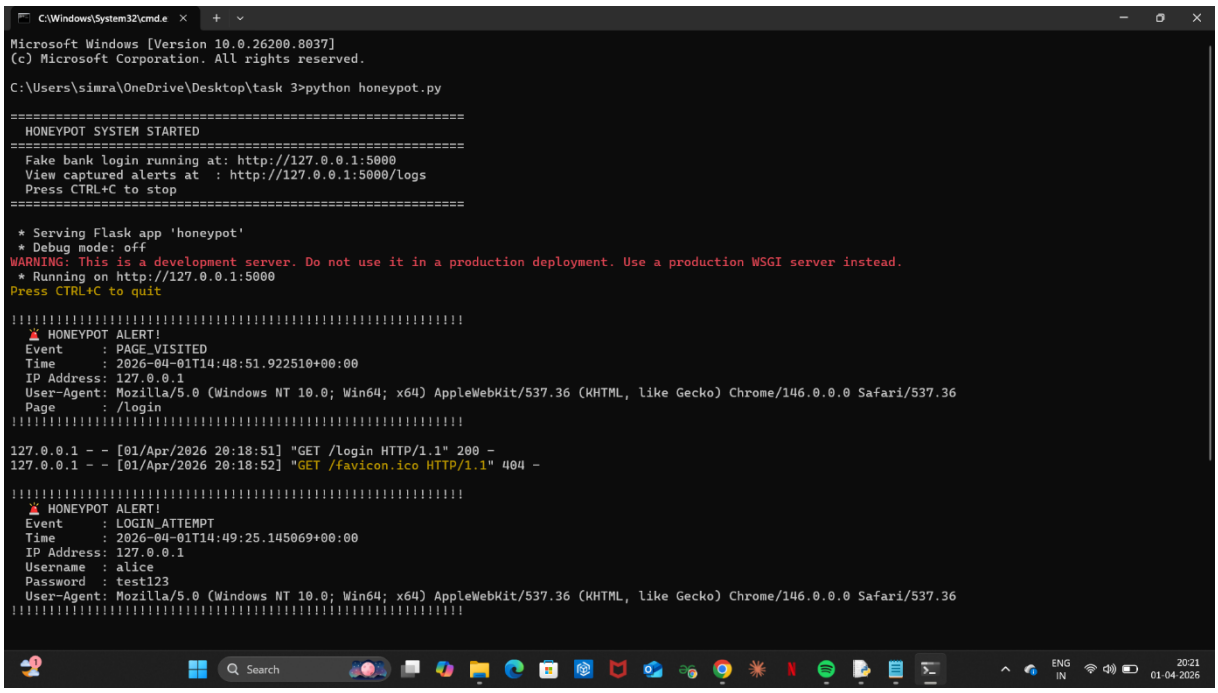
The application also recorded the login attempt in the honeypot log file at the same time. The log entry had important information like the username and password that were entered, the time and date of the attempt, the IP address, and the browser details. The JSON log file was able to save these records, and the administrative monitoring dashboard could show them.

The admin dashboard showed the events that were recorded, so admins could see suspicious interactions as they happened. The dashboard made it easy to see different types of events, like PAGE_VISITED and LOGIN_ATTEMPT, along with their timestamps and other information.

The testing results verified that the system could successfully:

- Identify visits to the fraudulent login page
- Capture and log attempts to log in
- Document detailed information regarding user interactions
- Present recorded events via the administrative dashboard

These findings illustrate that the honeypot system effectively tracks interactions with the misleading login interface and logs suspicious activities for subsequent examination.


C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.
C:\Users\simra\OneDrive\Desktop\task 3>python honeypot.py
==========
HONEYPOT SYSTEM STARTED
==========
Fake bank login running at: http://127.0.0.1:5000
View captured alerts at : http://127.0.0.1:5000/logs
Press CTRL+C to stop
==========
* Serving Flask app 'honeypot'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
!!!

Even though the system operates properly, it is still a basic prototype. In practical settings, added features like automated alert notifications, integration with intrusion detection systems, and sophisticated attacker behavior analysis could improve the efficiency of the honeypot system.

In summary, the results validate that the established honeypot system effectively showcases the idea of deception-based security by luring in questionable interactions and capturing them for observation and analysis.

6. Conclusion

This project effectively implemented a security mechanism based on deception through the use of a honeypot system. A counterfeit banking login page was created to serve as a bait for potential attackers, enabling the system to track and document any suspicious activities. The system records crucial information like IP addresses, user credentials, browser specifics, and timestamps each time a user engages with the login interface.

The testing results showed that the system successfully identifies visits to the fraudulent login page and logs login attempts as suspicious activities. All recorded interactions were organized in a structured log file and presented via an administrative dashboard, enabling administrators to observe potential attacker activities.

The project demonstrates how deceptive strategies can serve as an added layer of protection in cybersecurity systems. Honeypots can offer important insights into attack patterns and assist in enhancing defensive strategies by attracting and monitoring malicious activity. Even though the system in place is a basic prototype, it showcases the key concepts of deception-based security and honeypot technology. By adding features like automated alerts, integration with intrusion detection systems, and advanced monitoring capabilities, the system could evolve into a more sophisticated cybersecurity defense solution.

7. Refernces

Flask Documentation – https://flask.palletsprojects.com
Python Official Documentation – https://docs.python.org